Charles Cundiff

2/16/2026

AE 498

AE498 Github: https://github.com/Charles-Cundiff/AE498s

The homework assignment was to calculate the Minimum Orbit Intersection Distance (MOID) for a variety of objects. I employed Wisniowski's method detailed in "Fast Geometric Method for Calculating Accurate Minimum Orbit Intersection Distances" (2013). This is a numerical method that first coarsely scans for minima and then using tuning finds the MOID. Before scanning, however, the frame is rotated so that the first body's inclination, right ascension, and argument of periapsis are all zero the second body's angular parameters are changed.

The meridional plane is formed from the object B and the poles. During the coarse scan, both objects are assumed to lie in this plane and the position of object B is found using the equations below.

$$r_{B0} = \frac{a_B(1-e_B^2)}{1+e_B\cos\nu_0}$$

$$\begin{aligned} x_{B0} &= r_{B0}\left[\cos\Omega_B\cos(\omega_B+\nu_0) - \sin\Omega_B\sin(\omega_B+\nu_0)\cos i_B\right], \\ y_{B0} &= r_{B0}\left[\sin\Omega_B\cos(\omega_B+\nu_0) + \cos\Omega_B\sin(\omega_B+\nu_0)\cos i_B\right], \\ z_{B0} &= r_{B0}\sin(\omega_B+\nu_0)\sin i_B. \end{aligned}$$

As both objects are in the meridional plane, the tangent of object A's true anomaly is simply y/x. Object A's position is determined similarly to object B. Once all the cartesian locations are known, the distance between the objects is easily calculated.

$$r_A = a_A(1-e_A^2)/(1+e_A\cos L),$$

$$\begin{aligned} x_A &= r_A\cos L, \\ y_A &= r_A\sin L, \\ z_A &= 0. \end{aligned}$$

Once local minima are found on the meridional plane, tuning hones in on the true Moid. In tuning, slight deviations are added to the true anomalies of both objects. This creates six true anomalies: TA_1, TA_1L, TA_1R, TA_2, TA_2L, TA_2R. The distance is calculated between all nine perturbations. The lowest of these distances is the new baseline for the next step. The deviation reduces when no change is made until a minimum tuning size. Once the minimum tuning size is reached for each of the local minima, the MOID is easily found through comparison.

The code for this project was baselined from a Julia implementation of this paper's method linked here.

[mkretlow/MOID.jl: Compute the MOID - Minimum Orbit Intersection Distance for two given elliptical orbits (Kepler elements)]

The results from my code are below for each case

```
Earth-Apophis
    Min 0: TA = 2.1615039489638392 rad, L = 2.1615039489638392 rad, D = 0.005667839178089248 AU
    Min 1: TA = 4.257335434854402 rad, L = 4.257335434854402 rad, D = 0.052203508339746185 AU
    MOID: TA = 2.1615039489638392 rad, L = -2.233453017907201 rad, D = 0.005667839178089248 AU
Earth-YR4
    Min 0: TA = 0.8475775207424001 rad, L = 0.8475775207424001 rad, D = 0.05833984842582142 AU
    Min 1: TA = 5.456768521502725 rad, L = 5.456768521502725 rad, D = 0.0017677567485909576 AU
    MOID: TA = 5.456768521502725 rad, L = -0.20816531169280006 rad, D = 0.0017677567485909576 AU
Apophis-YR4
    Min 0: TA = 5.384235887042562 rad, L = 5.384235887042562 rad, D = 0.05033933734559084 AU
    Min 1: TA = 6.240264130355202 rad, L = 6.240264130355202 rad, D = 0.05524618573155603 AU
    MOID: TA = 5.384235887042562 rad, L = -2.169392779960321 rad, D = 0.05033933734559084 AU
Earth-Atlas
    Min 0: TA = 3.142044564479999 rad, L = 3.142044564479999 rad, D = 0.9037985777988469 AU
    Min 1: TA = 6.27936781824 rad, L = 6.27936781824 rad, D = 0.37841543299518077 AU
    MOID: TA = 6.27936781824 rad, L = -1.9675023471411213 rad, D = 0.37841543299518077 AU
```

They were compared against results from the Julia code and matched well.

```
PS C:\Classes\AE498\AE498\HW1> julia wisniowsky_julia.jl
MOID Earth-Apophis: (0.005667839178089219, -2.2334530149451544, 2.1615039521730193) (dist (au), ta1 (rad), ta2 (rad)
MOID Earth-Yr4: (0.0017677567485909756, -0.2081653118400187, 5.456768521654268) (dist (au), ta1 (rad), ta2 (rad)
MOID Apophis-Yr4: (0.050339337345590804, -2.169392788927703, 5.384235878018363) (dist (au), ta1 (rad), ta2 (rad)
MOID Earth-Atlas: (0.3784154329951802, -1.9675023766432198, 6.279367844669339) (dist (au), ta1 (rad), ta2 (rad)
```

Hyperbolic case accuracy

No alterations were made to the code for the hyperbolic case, but the Wisniowsky approach still seems to have worked. The JPL MOID for 3I Atlas is 0.364599 while mine is 0.3784. This small difference could easily be made up by JPL's more accurate models that which account for perturbations that mine does not. The Wisniowsky approach works without alteration because it will move towards the minimum naturally due to its tuning method. The coarse scan ensures it is tuning starts close enough to find the MOID.

My Code Explanation

This block extracts the Keplerian elements and sets the scanning/tuning steps. The coarse scan and initial tuning step sizes were taken from the paper. The final tuning step sizes were chosen for my desired accuracy.

```
a1, e1, i1, argp1, raan1 = kep1
a2, e2, i2, argp2, raan2 = kep2

# Step size constants
scanning_step = 0.12 # initial scanning step size (rad)
ini_tuning_step = 0.07 # initial tuning step size (rad)
fin_tuning_step = 1e-10 # final tuning size (rad)
```

This block creates frame rotations. The order is raan about axis 3, inclination about axis 1, then argp about axis 3.

```
# First orbit rotation matrix
R11 = np.cos(raan1) * np.cos(argp1) - np.sin(raan1) * np.cos(i1) * np.sin(argp1)
R12 = np.sin(raan1) * np.cos(argp1) + np.cos(raan1) * np.cos(i1) * np.sin(argp1)
R13 = np.sin(i1) * np.sin(argp1)
R21 = -np.cos(raan1) * np.sin(argp1) - np.sin(raan1) * np.cos(i1) * np.cos(argp1)
R22 = -np.sin(raan1) * np.sin(argp1) + np.cos(raan1) * np.cos(i1) * np.cos(argp1)
R23 = np.sin(i1) * np.cos(argp1)
R31 = np.sin(i1) * np.sin(raan1)
R32 = -np.sin(i1) * np.cos(raan1)
R33 = np.cos(i1)

# Second orbit rotation matrix
x1 = np.cos(raan2) * np.cos(argp2) - np.sin(raan2) * np.cos(i2) * np.sin(argp2)
x2 = np.sin(raan2) * np.cos(argp2) + np.cos(raan2) * np.cos(i2) * np.sin(argp2)
x3 = np.sin(i2) * np.sin(argp2)
y1 = -np.cos(raan2) * np.sin(argp2) - np.sin(raan2) * np.cos(i2) * np.cos(argp2)
y2 = -np.sin(raan2) * np.sin(argp2) + np.cos(raan2) * np.cos(i2) * np.cos(argp2)
y3 = np.sin(i2) * np.cos(argp2)
z1 = np.sin(i2) * np.sin(raan2)
z2 = -np.sin(i2) * np.cos(raan2)
z3 = np.cos(i2)
```

This block of code finds the new inclination, raan, and argp from the new x, y, z axes.

```
# Values needed to calculate new i, raan, w
z1n = R11 * z1 + R12 * z2 + R13 * z3
z2n = R21 * z1 + R22 * z2 + R23 * z3
z3n = R31 * z1 + R32 * z2 + R33 * z3
y3n = R31 * y1 + R32 * y2 + R33 * y3
x3n = R31 * x1 + R32 * x2 + R33 * x3

# New inclination, raan, and arg peri
i2 = np.arctan2(np.sqrt(z1n * z1n + z2n * z2n), z3n)
raan2 = -np.arctan2(z1n, -z2n)
argp2 = -np.arctan2(x3n, y3n)
i1, raan1, argp1 = 0, 0, 0
```

This block of code is the distance function used throughout the minimization. It employs the equations reviewed earlier for known TA1 (L) and TA2 (TA).

```
# distance function to be used later
def dist(L, TA):
    """
    Finds distance between orbit one and two given their true anomalies
    Inputs:
        L: true anomaly of first orbit (rad)
        TA: true anomaly of second orbit (rad)
    Output:
        Distance between objects
    """
    r1 = a1 * (1 - e1**2) / (1 + e1 * np.cos(L))
    r2 = a2 * (1 - e2**2) / (1 + e2 * np.cos(TA))

    x1 = r1 * np.cos(L)
    y1 = r1 * np.sin(L)
    z1 = 0

    x2 = r2 * (np.cos(raan2) * np.cos(argp2 + TA) - np.sin(raan2) * np.sin(argp2 + TA) * np.cos(i2))
    y2 = r2 * (np.sin(raan2) * np.cos(argp2 + TA) + np.cos(raan2) * np.sin(argp2 + TA) * np.cos(i2))
    z2 = r2 * (np.sin(argp2 + TA) * np.sin(i2))

    D = np.sqrt((x1 - x2)**2 + (y1 - y2)**2 + (z1 - z2)**2)

    return D
```

This block of code completes the coarse scan. It tracks the list of the last three distances, and true anomalies of both objects. If the middle distance is less than the other two, then it is marked as a local min. Because of this implementation, the iteration goes 3 steps after 2pi.

```python
# ---------------------------------------
# Step 1: scanning along meridional plane
# ---------------------------------------
ta2 = 0
d_min_list = [] # list of distances
ta_min_list = [] # list of true anomalies of second orbit
l_min_list = [] # list of longitudes of first orbit

d0_triple = np.array([0, 0, 0]) # list of distances to decide on local min
ta_triple = np.array([0, 0, 0])
l_triple = np.array([0, 0, 0])
def update_triple(val, list):
    list[2] = list[1]
    list[1] = list[0]
    list[0] = val

while ta2 < 2 * np.pi + 3 * scanning_step:
    # calculate position of second orbit
    r20 = a2 * (1 - e2**2) / (1 + e2 * np.cos(ta2))
    x20 = r20 * (np.cos(raan2) * np.cos(argp2 + ta2) - np.sin(raan2) * np.sin(argp2 + ta2) * np.cos(i2))
    y20 = r20 * (np.sin(raan2) * np.cos(argp2 + ta2) + np.cos(raan2) * np.sin(argp2 + ta2) * np.cos(i2))
    l0 = np.atan2(y20, x20)

    # calculate distance between orbits
    d0 = dist(l0, ta2)

    # check for local minimum
    update_triple(d0, d0_triple)
    update_triple(ta2, ta_triple)
    update_triple(l0, l_triple)
    if d0_triple[1] < d0_triple[0] and d0_triple[1] < d0_triple[2]:
        d_min_list.append(d0_triple[1])
        ta_min_list.append(ta_triple[1])
        l_min_list.append(l_triple[1])

    # move scan forward
    ta2 = ta2 + scanning_step

d_min_list = np.array(d_min_list, dtype = float)
ta_min_list = np.array(ta_min_list, dtype = float)
l_min_list = np.array(l_min_list, dtype = float)
```

This “water procedure” will occur if there is only one minimum found in coarse scanning. It will add four evenly spaced starting points to ensure the MOID is found. This did not activate during the homework assignment.

```python
# Water procedure: distributes points evenly if only one min exists
if ta_min_list.size == 1:
    ta_min_list = np.array([np.pi / 8, 5 * np.pi / 8, 9 * np.pi / 8, 13 * np.pi / 8])
    for i in range(ta_min_list.size):
        r20 = a2 * (1 - e2**2) / (1 + e2 * np.cos(ta_min_list[i]))
        x20 = r20 * (np.cos(raan2) * np.cos(argp2 + ta_min_list[i]) - np.sin(raan2) * np.sin(argp2 + ta_min_list[i]) * np.cos(i2))
        y20 = r20 * (np.sin(raan2) * np.cos(argp2 + ta_min_list[i]) + np.cos(raan2) * np.sin(argp2 + ta_min_list[i]) * np.cos(i2))
        l0 = np.atan2(y20, x20)

        # calculate distance between orbits
        d0 = dist(l0, ta_min_list[i])
        l_min_list[i] = l0
        d_min_list[i] = d0
```

This tuning block of code conducts the parallel tuning algorithm described in the paper. It is not very efficient because it recalculates many distances of repeated TA-L combinations. Nonetheless, it works fast enough for this homework assignment.

```
# --------------------
# Step 2: tuning
# --------------------
for i in range(ta_min_list.size):
    tuning_step = ini_tuning_step
    TA = ta_min_list[i]
    L = l_min_list[i]
    D = d_min_list[i]
    while tuning_step > fin_tuning_step:
        not_finished = True
        D_arr = np.array([0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0])
        while not_finished:
            L_l = (L - tuning_step)
            L_r = (L + tuning_step)
            TA_l = (TA - tuning_step)
            TA_r = (TA + tuning_step)

            TA_arr = np.array([TA, TA_l, TA_r, TA, TA_l, TA_r, TA, TA_l, TA_r])
            L_arr = np.array([L, L, L, L_l, L_l, L_l, L_r, L_r, L_r])
            D_arr[0] = dist(L, TA)
            D_arr[1] = dist(L, TA_l)
            D_arr[2] = dist(L, TA_r)
            D_arr[3] = dist(L_l, TA)
            D_arr[4] = dist(L_l, TA_l)
            D_arr[5] = dist(L_l, TA_r)
            D_arr[6] = dist(L_r, TA)
            D_arr[7] = dist(L_r, TA_l)
            D_arr[8] = dist(L_r, TA_r)

            min_idx = np.argmin(D_arr)
            # potential print functions to see each step of tuning
            # print(f"Tuning: TA = {TA}, L = {L}, D = {D / 1.496e+8}, min_idx = {min_idx}")
            # print(f"D_arr: {D_arr / 1.496e+8}")
            TA = TA_arr[min_idx]
            L = L_arr[min_idx]
            D = D_arr[min_idx]

            if min_idx == 0:
                not_finished = False
        tuning_step = tuning_step * 0.2
        ta_min_list[i] = TA
        l_min_list[i] = L
        d_min_list[i] = D
```

This block simply selects the MOID and prints it.

```python
    for i in range(ta_min_list.size):
        print(f"    Min {i}: TA = {ta_min_list[i]} rad, L = {ta_min_list[i]} rad, D = {d_min_list[i] / AU} AU")
    moid_idx = np.argmin(d_min_list)
    print(f"    MOID: TA = {ta_min_list[moid_idx]} rad, L = {l_min_list[moid_idx]} rad, D = {d_min_list[moid_idx] / AU} AU")
```